

**UNIVERSITY COLLEGE OF ENGINEERING  
BIT CAMPUS  
ANNA UNIVERSITY**

**IBM AGENTIC AI INTERNSHIP**

*Internship Project Report*

**AI IT HELPDESK AGENT**

*(An Agentic AI-Based Web Application for Automated IT Support)*

Submitted by

**Niranjan R**

Register No: 810023205002

2026

# 1. Project Overview

---

## 1.1 Project Title

AI IT Helpdesk Agent — A Web-Based Agentic AI Application for Automated IT Support.

## 1.2 Problem Statement

Organizations of every size receive a continuous stream of repetitive IT support requests — Wi-Fi and network failures, slow or freezing computers, printer errors, email/Outlook issues, forgotten passwords, and software installation problems. Traditional helpdesk workflows depend on human support staff to triage, diagnose, and resolve each request manually, which leads to long response times, inconsistent troubleshooting quality, and support teams being overwhelmed by low-complexity, repetitive tickets. Employees are often left waiting for even basic guidance that could be resolved instantly with the right information. There is a clear need for an intelligent, always-available assistant that can understand an employee's problem in natural language, provide accurate troubleshooting steps immediately, and automatically escalate genuinely serious issues into a trackable support ticket.

## 1.3 Brief Description of the Project

The AI IT Helpdesk Agent is a self-contained, locally hosted web application that acts as a first-line IT support assistant. It combines a clean HTML/CSS/JavaScript front end with a Python Flask backend and a locally running open-source large language model (Mistral, served through Ollama) to hold natural, conversational troubleshooting sessions with users. The agent automatically analyzes each incoming query to detect its category (Network, Hardware, Software, Email, Account, or Printer) and urgency, draws on a built-in searchable knowledge base of common IT fixes to ground its answers, and — for issues that need human follow-up — can instantly generate a structured support ticket stored in a local SQLite database. Because the entire pipeline (model inference, data storage, and diagnostics) runs on the local machine, no company data or employee query ever leaves the system, making it a privacy-preserving alternative to cloud-based helpdesk AI tools.

# 2. Objectives & Proposed Solution

---

## 2.1 Project Objectives

- To build a conversational AI assistant that can diagnose and resolve common IT problems without human intervention.
- To reduce the response time and workload on human IT support staff by automating first-level troubleshooting.
- To maintain complete data privacy by running the AI model and database entirely on the local machine, with no external API calls.
- To provide a searchable, categorized knowledge base that both employees and the AI can reference.
- To implement an end-to-end support ticket lifecycle — creation, tracking, prioritization, and status updates.
- To present real-time system diagnostics (CPU, RAM, disk, connectivity) to help pinpoint hardware-related issues.

- To design an intuitive, single-page web interface that requires no installation on the end user's side beyond a browser.

## 2.2 How the Agentic AI Solution Works

The application follows an agentic workflow rather than a simple question-and-answer chatbot. When a user submits a message through the chat interface, the Flask backend first passes the query to a lightweight rule-based classifier that scans for keywords to automatically detect the issue's category (e.g., Network, Hardware, Printer) and urgency level (Low, Medium, Critical). In parallel, the system searches the knowledge base for contextually relevant articles matching the query and injects that verified information into the prompt sent to the Mistral model, ensuring the AI's response is grounded in accurate, company-relevant troubleshooting steps rather than relying purely on the model's general knowledge. The AI then returns a clear, numbered set of troubleshooting instructions. If the detected issue appears serious — such as a security concern, data loss, or a device being completely unusable — the agent proactively recommends creating a support ticket, and can pre-fill the ticket's title, category, priority, and description directly from the conversation, minimizing manual data entry for the user.

## 2.3 Key Features

- **AI-Powered Chat:** Natural-language troubleshooting conversation using the local Mistral model via Ollama.
- **Automatic Issue Detection:** Keyword-driven classification of category and priority for every query.
- **Knowledge-Base-Grounded Answers:** AI responses are backed by a curated local knowledge base to improve accuracy.
- **Ticket Management System:** Create, view, update, and track IT support tickets with auto-generated codes (e.g., IT-1001).
- **One-Click Ticket Creation from Chat:** Converts an AI conversation directly into a pre-filled support ticket.
- **Searchable Knowledge Base:** Categorized articles covering Network, Hardware, Software, Email, Account, and Printer issues.
- **Urgent Help Flow:** A dedicated path for logging Critical-priority issues immediately.
- **Live Dashboard Statistics:** Real-time counts of Open, In Progress, and Resolved tickets.
- **Read-Only System Diagnostics:** Displays CPU, RAM, disk usage, OS details, and internet connectivity status.
- **Graceful Degradation:** Clearly informs the user if the local AI model is unavailable, instead of crashing.

# 3. Implementation & Results

---

## 3.1 Technologies / Tools Used

- **Frontend:** HTML5, custom CSS (purple/indigo theme), vanilla JavaScript (no external frameworks).
- **Backend:** Python 3, Flask (REST API framework).
- **AI Engine:** Ollama (local AI model runner) running the open-source Mistral large language model.
- **Database:** SQLite (lightweight, file-based relational database for ticket storage).

- System Utilities: psutil (for read-only CPU, RAM and disk diagnostics).

### 3.2 Working Process

1. The Flask backend (app.py) initializes the SQLite database and serves the single-page front end (templates/index.html).
2. The user opens the web application in a browser and interacts through views such as Home, Ask AI, Tickets, Knowledge Base, Urgent Help, and Settings.
3. When the user sends a message, static/app.js issues a POST request to the /api/chat endpoint with the message and recent chat history.
4. ai\_engine.py checks whether Ollama and the Mistral model are available, retrieves relevant knowledge-base context via knowledge\_base.py, and forwards the combined prompt to the local Mistral model.
5. The AI's reply and a rule-based analysis (detected category and priority) are returned as JSON and rendered in the chat window.
6. If the user chooses to raise a ticket, the front end pre-fills the ticket form from the AI's analysis and submits it to /api/tickets, which database.py stores in SQLite with an auto-generated ticket code.
7. Users can browse or search the knowledge base via /api/kb, and administrators can view or update ticket status/priority via /api/tickets/<code>.
8. The Settings view calls /api/diagnostics, which uses diagnostics.py and psutil to safely report CPU, RAM, disk, and connectivity information without executing or modifying anything on the system.

The complete list of REST API endpoints exposed by the Flask backend, which the JavaScript front end consumes to power every view, is summarized below.

- GET /api/status — Check Ollama / Mistral model availability.
- POST /api/chat — Send a user message and receive an AI-generated reply.
- GET /api/kb — Search / filter the knowledge base articles.
- GET /api/kb/<id> — Fetch a single knowledge base article.
- GET /api/tickets — List all support tickets.
- POST /api/tickets — Create a new support ticket.
- PATCH /api/tickets/<code> — Update ticket status or priority.
- GET /api/stats — Get open / in-progress / resolved ticket counts.
- GET /api/diagnostics — Return read-only system diagnostic information.

### 3.3 Screenshots / Output

The application's single-page interface presents the AI chat, quick troubleshooting shortcuts, a live overview of ticket statistics, and a searchable knowledge base on the Home view, with dedicated Ask AI, Tickets, Knowledge Base, Urgent Help, and Settings sections accessible from the navigation. (Insert screenshots of the running application — Home, Ask AI chat, Tickets list, and Settings/Diagnostics views — here before final submission.)

### 3.4 Results Achieved

- A fully working local AI helpdesk that answers common IT queries with step-by-step troubleshooting instructions in real time.
- Automatic categorization and priority-tagging of user issues with no manual configuration required.
- A functioning ticketing system with auto-incrementing ticket codes and full status/priority lifecycle tracking, backed by SQLite.
- A searchable knowledge base of six IT categories that improves the accuracy and relevance of AI responses.
- Live, read-only system diagnostics (CPU, RAM, disk, OS, connectivity) integrated directly into the web interface.
- Complete data privacy — all AI inference and storage occurs on the local machine with no cloud dependency.
- A responsive, framework-free front end that loads quickly and requires no build tooling.

## 4. Conclusion & Future Scope

---

### 4.1 Project Conclusion

The AI IT Helpdesk Agent successfully demonstrates how an agentic AI workflow — combining a locally hosted large language model, a curated knowledge base, rule-based issue analysis, and an automated ticketing pipeline — can meaningfully reduce the burden of first-level IT support. By keeping every component (AI inference, data storage, and diagnostics) on the local machine, the project achieves both intelligent automation and strong data privacy, making it well suited for organizations that are cautious about sending internal support data to third-party cloud services. The project validates the practical use of open-source local AI models for real-world enterprise support scenarios.

### 4.2 Challenges Faced

- Ensuring the AI model produced consistent, well-structured troubleshooting steps rather than overly generic or verbose answers.
- Designing a lightweight, dependency-free keyword classifier that could reliably infer issue category and priority without a second AI call.
- Handling graceful degradation when the local Ollama server or the Mistral model was not running, so the interface remained usable instead of crashing.
- Keeping the knowledge base context concise enough to fit within the model's prompt while still improving answer accuracy.
- Coordinating state between the front-end chat session (held in browser memory) and backend ticket creation so that AI-detected details were carried over correctly.

### 4.3 Future Enhancements

- Persist chat history in the database so conversations survive page refreshes and can be reviewed later.
- Add user authentication and role-based access so employees and IT administrators have separate views and permissions.

- Introduce analytics dashboards showing trends in issue categories, resolution times, and recurring problems.
- Support file/screenshot uploads within the chat so users can attach error screenshots for better diagnosis.
- Add email or messaging notifications when a ticket's status changes.
- Allow the knowledge base to be edited through an admin interface instead of being hard-coded in Python.
- Explore fine-tuning or retrieval-augmented generation (RAG) with a vector database for more precise knowledge-base grounding.
- Containerize the application (Docker) for easier deployment across different organizational environments.

## 4.4 References

1. Ollama Documentation — <https://ollama.com/>
2. Mistral AI — Open-Source Large Language Model — <https://mistral.ai/>
3. Flask Documentation — <https://flask.palletsprojects.com/>
4. SQLite Documentation — <https://www.sqlite.org/docs.html>
5. psutil Documentation — <https://psutil.readthedocs.io/>
6. Project source code and README: ai\_it\_helpdesk\_web (internship project repository).